

مرتب‌سازی (Sortieren)

IHK به سبک (Pseudocode) شبه‌کد

Algorithmus sortiereArray(A: Array von Integer):
N ← A.length
für i ← 0 bis N-2:
minIndex ← i
für j ← i+1 bis N-1:
wenn A[j] < A[minIndex]

(Pseudo-Java) نسخه شبه‌جاوا

```
public static void sortiereArray(int[] A) {
    int N = A.length;
    for (int i = 0; i < N - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j
```

توضیح الگوریتم به فارسی

یکی از متداول‌ترین موضوعات در امتحانات IHK، الگوریتم‌های **مرتب‌سازی** (*Sortieren*) است. در صورت سؤال، معمولاً عباراتی نظیر «مرتب‌سازی صعودی/نزولی یک آرایه» یا «چیدمان داده‌ها بر اساس ...» دیده می‌شود که نشان می‌دهد نیاز به پیاده‌سازی یک الگوریتم مرتب‌سازی دارید.
راحتل استاندارد، استفاده از یکی از الگوریتم‌های مرتب‌سازی کلاسیک (مثل *Bubble Sort*، *Selection Sort*، *Insertion Sort* و غیره) است. در مثال فوق، از الگوریتم **مرتب‌سازی انتخابی** (**Selection Sort**) برای مرتب‌سازی یک آرایه اعداد صحیح به ترتیب صعودی استفاده شده است. این الگوریتم در هر مرحله کوچک‌ترین مقدار باقیمانده در بخش نامرتب آرایه را پیدا کرده و آن را با عنصر جاری تعویض می‌کند تا آن مقدار در جای درست خود قرار گیرد.

در پاسخ به چنین سؤالی، ابتدا باید تشخیص دهید که مسئله مرتبط با مرتب‌سازی است. کلمات کلیدی مانند «Sortieren» یا توضیحاتی درباره مرتب کردن داده‌ها راهنمای شما هستند. سپس الگوریتم مناسب را انتخاب کنید (در امتحان معمولاً اشاره نمی‌شود که کدام مرتب‌سازی را به کار ببرید، بلکه «بیک الگوریتم مرتب‌سازی به انتخاب خود» در خواست می‌شود). بسیار مهم است که نام الگوریتم انتخابی خود را ذکر کنید (برای مثال: مرتب‌سازی انتخابی) زیرا اغلب در صورت سؤال صراحتاً می‌خواهند که مشخص کنید از چه روشی استفاده کرده‌اید.

در شبه‌کد ارائه‌شده، از متغیر minIndex برای نگهداری موقعیت کوچک‌ترین عنصر در بخش باقیمانده آرایه استفاده شده است. دو حلقه تو در تو تمام عناصر را مرور می‌کنند؛ حلقه بیرونی هر بار یک عنصر را در جای درست می‌نشاند و حلقه درونی کوچک‌ترین عنصر را در بخش باقی‌مانده پیدا می‌کند. در نهایت، عمل تعویض (سواپ) انجام می‌شود تا عنصر کوچک‌تر در جایگاه i قرار گیرد. نسخه شبه‌جاوای همان الگوریتم نیز به وضوح همان منطق را با سینتکس مشابه به کد Java نشان می‌دهد. در امتحان، نوشتن شبه‌کد به این صورت (شباهت به کدنویسی معمول، با استفاده از ساختارهای کنترلی واضح مانند for و if) مورد قبول است.

آرایه‌های دوبعدی (Terminplan, Besucher zählen)

IHK به سبک (Pseudocode) شبه‌کد

Algorithmus findeBesucherreichstenTag(matrix: Array[0..T-1][0..H-1] von Integer):
Integer maxSumme ← -∞
tagIndex ← -1
برای t ← 0 تا T-1:

(Pseudo-Java) نسخه شبه‌جاوا

```
public static int findeBesucherreichstenTag(int[][] matrix) {
    int T = matrix.length;
    int H = matrix[0].length;
    int maxSumme = Integer.MIN
```

توضیح الگوریتم به فارسی

در بسیاری از سؤالات امتحانی IHK، کار با **آرایه‌های دوبعدی** مطرح می‌شود. این نوع سوالات ممکن است در قالب یک جدول زمان‌بندی (*Terminplan*) یا شمارش داده‌ها در یک جدول (مثلاً شمارش بازدیدکنندگان - *Besucher zählen*) باشند.
کلید حل این سؤالات، تشخیص آرایه دوبعدی در صورت سؤال و نیاز به استفاده از **حلقه‌های تو در تو** است. به عنوان مثال، اگر جدولی از اعداد (مثلاً تعداد بازدیدکنندگان در ساعات مختلف طی چند روز) داده شود و از شما خواسته شود که روزی را با بیشترین بازدیدکننده پیدا کنید، الگوریتم باید ابتدا جمع بازدیدکنندگان هر سطر (هر روز) را محاسبه کند و سپس بیشترین آنها را تشخیص دهد.

در قطعه شبه‌کد بالا، matrix نمایانگر یک آرایه دوبعدی است که مثلاً T سطر (روز) و H ستون (ساعت یا بازه‌های زمانی) دارد. حلقه بیرونی روی سطرها (روزها) می‌چرخد و برای هر روز، یک متغیر summe برای جمع‌آوری تعداد بازدیدکنندگان آن روز استفاده می‌شود. حلقه داخلی تمامی ستون‌های آن سطر را مرور کرده و مقادیر را جمع می‌زند. سپس مجموع هر روز با حداکثر مجموع فعلی (maxSumme) مقایسه می‌شود تا در صورت بیشتر بودن، آن روز به عنوان روز با بیشترین بازدیدکننده ثبت شود. در پایان، شاخص مربوط به آن روز (tagIndex) برگردانده می‌شود.

سوالاتی از این دست ممکن است شکل‌های مختلفی داشته باشند؛ برای مثال پیدا کردن اولین زمان خالی در یک جدول زمان‌بندی (که نیاز به بررسی مقادیر خالی یا 0 در آرایه دوبعدی دارد) یا محاسبه مجموع یک ستون مشخص در یک ماتریس. اما در همه این موارد، ایده اصلی استفاده از دو حلقه تو در تو برای پیمایش ساختار دوبعدی است. در امتحان، به خوانایی شبه‌کد توجه کنید: اندیس‌ها و اندازه آرایه را درست به کار ببرید و حتماً مشخص کنید که چگونه به عناصر آرایه دوبعدی دسترسی می‌گیرید (مثلاً matrix[t][h]). نسخه شبه‌جاوای ارائه‌شده نیز همان منطق را در قالب سینتکس آشنای Java نمایش می‌دهد.

(vergleicheMitarbeiter) تابع مقایسه اشیاء

IHK به سبک (Pseudocode) شبه‌کد

Algorithmus vergleicheMitarbeiter(m1: Mitarbeiter, m2: Mitarbeiter):
Integer m1.gehalt < m2.gehalt اگر 1-مأخرت اگر

(Pseudo-Java) نسخه شبه‌جاوا

```
public static int vergleicheMitarbeiter(Mitarbeiter m1, Mitarbeiter m2) {
    if (m1.getGehalt() < m2.getGehalt()) {
        return -1;
    } else if (m
```

توضیح الگوریتم به فارسی

برخی از سوالات الگوریتمی در امتحانات IHK مربوط به نوشتن **تابع مقایسه** برای دو شیء هستند. برای مثال، تابع *vergleicheMitarbeiter* که دو شیء از کلاس Mitarbeiter (کارمند) را با هم مقایسه می‌کند. معمولاً این گونه توابع به سبک *Comparator* در جاوا عمل می‌کنند و یک عدد صحیح برمی‌گردانند که نشان‌دهنده ترتیب مقایسه است: مقدار منفی به معنی «اولی کوچکتر از دومی»، مقدار مثبت به معنی «اولی بزرگتر از دومی» و صفر به معنی «برابر بودن یا ترتیب یکسان».

در صورت سؤال امتحانی ممکن است به صراحت بیان شود که تابع مقایسه باید چگونه کار کند. برای مثال: «تابع *vergleicheMitarbeiter* مقداری بزرگتر از 0 برمی‌گرداند اگر حقوق کارمند دوم باید قبل از اولی مرتب شود». این عبارت معادل آن است که اگر حقوق (*Gehalt*) کارمند دوم بیشتر است، ترتیب مورد نظر ایجاب می‌کند دومین جلوتر بیاید، لذا تابع عدد مثبتی برمی‌گرداند. در الگوریتم نمونه ما، برای سادگی فرض شده که معیار مقایسه، حقوق کارکنان است (متغیر فرضی *gehalt*). به این ترتیب، اگر حقوق m1 کمتر از m2 باشد مقدار -1 برمی‌گردد (m1 کوچکتر است)، اگر بیشتر باشد 1 (m1 بزرگتر است) و در غیر این صورت 0 (حقوق برابر یا در بحث مرتب‌سازی، هیچ‌کدام بر دیگری برتری ندارد).

کلید تشخیص چنین سؤالی، مشاهده کلماتی مثل *vergleiche*...() یا توضیحاتی در صورت سؤال درباره مقایسه دو شیء است. در پاسخ، باید ساختار یک تابع مقایسه را بنویسید. معمولاً نوع بازگشتی Integer (یا int) در نظر گرفته می‌شود و توجه داشته باشید که حتماً هر سه حالت کمتر، بیشتر و برابر را پوشش دهید. هم در شبه‌کد و هم در نسخه شبه‌جاوا، رعایت ساختار شرطی چندگانه (if/else if/else) ضروری است تا پاسخ کامل باشد.

تبدیل عدد به باینری یا محاسبه رقم کنترلی

تبدیل ددهی به دودویی – (IHK به سبک Pseudocode) شبه‌کد

Algorithmus dezimalZuBinaer(n: Integer): String
اگر n == 0
"0" برگردان
ergebnis ← ""
// رشته خالی مادی‌می که
n > 0: bit ← n mod 2
e

تبدیل ددهی به دودویی – (Pseudo-Java) نسخه شبه‌جاوا

```
public static String dezimalZuBinaer(int n) {
    if (n == 0) return "0";
    String result = "";
    while (n > 0) {
        int bit = n % 2;
        result = bit
```

توضیح الگوریتم به فارسی

در بخش دیگری از سوالات پرتکرار، ممکن است از شما خواسته شود که یک عدد را **تبدیل به مبنای دیگر** کنید (مثلاً تبدیل عدد ددهمی به معادل دودویی آن) یا یک **رقم کنترلی (Prüfziffer)** را محاسبه کنید. کلید تشخیص این گونه سوالات، مشاهده عباراتی مثل «تبدیل عدد» یا «محاسبه رقم کنترل/صحیح» در متن سوال است. برای تبدیل مبنأ، معمولاً الگوریتم کلاسیک تقسیم متوالی عدد بر مبنای مورد نظر (مثلاً 2 برای دودویی) و جمع‌آوری باقیمانده‌ها به عنوان رقم‌های مبنای جدید به کار می‌رود. در مثال بالا، تابع dezimalZuBinaer عدد صحیح را به رشته متناظر در مبنای 2 تبدیل می‌کند. با هر بار تقسیم n بر ۲، باقی‌مانده (0 یا 1) به ابتدای رشته نتیجه اضافه می‌شود تا ترتیب ارقام دودویی صحیح باشد.

اگر سوال به جای تبدیل دودویی، به محاسبه‌ی *Prüfziffer* اشاره کند، باید دقت کنید که معمولاً روش محاسبه در صورت سوال توضیح داده شده است (برای مثال: جمع وزنی ارقام و گرفتن باقیمانده تقسیم بر 10، یا الگوریتم لو هن برای کارت‌های اعتباری). در چنین مواردی، الگوریتم کلی این است که از روی رقم‌های داده‌شده محاسبات خواسته‌شده را انجام دهید و رقم کنترلی را بدست آورید. مثلاً اگر گفته شود «رقم کنترلی برابر است با باقی‌مانده مجموع ارقام در 9»، شما باید حلقه‌ای بنویسید که تمام ارقام عدد را جمع کند و سپس mod 9 بگیرد. نکته مهم این است که صورت سوال را به دقت بخوانید و فرمول یا روال داده‌شده برای Prüfziffer را دقیقاً پیاده‌سازی کنید.

در هر دوی این موارد (تبدیل باینری یا Prüfziffer)، داشتن یک حلقه برای پیمایش رقم‌های عدد ضروری است. در تبدیل باینری، این کار با تقسیم پیاپی بر 2 انجام می‌شود تا زمانی که عدد صفر شود. در محاسبه رقم کنترلی نیز معمولاً باید یک حلقه روی تمام ارقام یا کاراکترهای ورودی داشته باشید. پاسخ خود را در قالب شبه‌کد باید مرحله به مرحله و واضح بنویسید (مثلاً ابتدا مقداردهی اولیه متغیر نتیجه، سپس حلقه و در نهایت خروجی). نسخه شبه‌جاوا نیز می‌تواند برای درک بهتر همان منطق ارائه شود، اما مهم تطابق آن با شبه‌کد منطقی شماس‌ت.

List<T> und Generics) کار با لیست‌ها و نوع‌های جنریک

IHK به سبک Pseudocode) شبه‌کد

در e برای هر عنصر liste[0] ← max ← Null/Nil یا مقدار
Nichts // است
آنگاه برگردان
listeleer اگر T
Algorithmus findeMaximum(liste: List<T>): T

Pseudo-Java) نسخه شبه‌جاوا

```
public static <T extends Comparable<T>> T findeMaximum(List<T> liste) {
    if (liste.isEmpty()) return null;
    T max = liste.get(0);
    for (T e
```

توضیح الگوریتم به فارسی

استفاده از **لیست‌ها** (مانند List<T> در جاوا) و مفاهیم **نوع‌های پارامتریک (Generics)** نیز از موضوعاتی است که ممکن است در امتحانات برنامه‌نویسی IHK مطرح شود. در چنین سوالاتی ممکن است از شما خواسته شود که الگوریتمی بنویسید که روی یک لیست با عناصر از نوع عمومی T کاری انجام دهد. کلید تشخیص این سوالات، دیدن نمادهایی مثل «<T> یا کلمه "Generic" در صورت سوال است. برای مثال، ممکن است سوال بپرسد: «تابعی بنویسید که بزرگ‌ترین عنصر یک List<T> را برمی‌گرداند» یا «الگوریتمی برای ادغام دو لیست Generic ارائه دهید».

در شبه‌کد نمونه بالا، تابع findeMaximum عمومی نوشته شده است، به این معنا که می‌تواند لیستی از هر نوع T را که قابل مقایسه باشد پردازش کند. ابتدا چک می‌کنیم که لیست خالی نباشد (این یک گام مهم در برنامه‌نویسی واقعی و همچنین نشان‌دهنده دقت شما در الگوریتم است). سپس فرض می‌کنیم اولین عنصر لیست بزرگ‌ترین مقدار فعلی است و با پیمایش لیست، هر عنصر را با max مقایسه می‌کنیم. اگر عنصری بزرگ‌تر بود، آن را به عنوان max جدید در نظر می‌گیریم. در پایان، max بزرگ‌ترین عنصر است بازگردانده می‌شود. در نسخه شبه‌جاوا، از ویژگی <T extends Comparable<T>> استفاده شده تا محدود کنیم نوع T قابلیت مقایسه داشته باشد (متد compareTo را داشته باشد). این دقیقاً معادل همان شرط e > max در شبه‌کد است.

سوالات مرتبط با لیست و نوع جنریک می‌توانند شامل موارد دیگری نیز باشند؛ مثلاً «پیاده‌سازی یک کلاس Generic که یک ساختار داده (مانند پشته یا صف) را مدیریت می‌کند» یا «تابعی که دو لیست از نوع T را گرفته و لیست سومی شامل ترکیب آنها برمی‌گرداند». در همه این موارد، مهم است که نحوه استفاده از نوع‌های عمومی را نشان دهید. برای مثال، اگر نام کلاس با تابع Generic داده شده (مثل <MeineListe>T) حتماً در جواب خود همان را منعکس کنید. همچنین روش دسترسی و پیمایش عناصر لیست (مثلاً با حلقه for each یا اندیس) باید واضح بیان شود. هدف این سوالات ارزیابی درک شما از ساختارهای داده داینامیک (لیست‌ها) و مفاهیم نوع‌بندی عمومی در برنامه‌نویسی است.

(Komprimierung) الگوریتم فشرده‌سازی داده‌ها

IHK به سبک Pseudocode) شبه‌کد

Algorithmus komprimiere(text: String): String
komprimiert ← ""
// رشته نتیجه
n ← länge(text)
i ← 0
aktuellesZeichen ← t
i < n:

Pseudo-Java) نسخه شبه‌جاوا

```
public static String komprimiere(String text) {
    StringBuilder result = new StringBuilder();
    int n = text.length();
    int i = 0;
    while (i <
```

توضیح الگوریتم به فارسی

آخرین بخش مورد نظر، الگوریتم‌های **فشرده‌سازی ساده** (مانند *Run-Length Encoding*) است که در سوالات امتحانی با عنوان *Komprimierung* شناخته می‌شوند. ایده‌ی کلی این الگوریتم‌ها کاهش حجم داده با استفاده از الگوهای تکراری است. ساده‌ترین شکل آن که در امتحانات مطرح می‌شود، **فشرده‌سازی رشته با شمارش تکرار حروف** است. به عنوان مثال، رشته‌ای مانند AAAABBC را در نظر بگیرید؛ خروجی فشرده‌سازی می‌تواند به صورت 4A2B1C باشد که نشان‌دهنده ۴ تا 'A'، سپس ۲ تا 'B' و ۱ 'C' است.

در صورت سوال امتحان ممکن است قالب خاصی برای خروجی تعیین شده باشد (مثلاً استفاده از یک کاراکتر خاص برای اعلان تکرار، نظیر علامت # یا چیزی شبیه مثال «4Q\$»). مهم است که این جزئیات را از صورت سوال استخراج کنید. در الگوریتم عمومی که در شبه‌کد بالا آمده است، ما به شکل ساده هرگاه یک کاراکتر بیش از یک‌بار تکرار شده باشد، تعداد تکرار را قبل از آن کاراکتر در رشته خروجی درج می‌کنیم. الگوریتم با استفاده از یک اشاره‌گر i روی کاراکترهای رشته ورودی جلو می‌رود. برای هر کاراکتر، ابتدا یک count روی 1 تنظیم می‌شود و سپس با یک حلقه در حالی که شمارش می‌کنیم چند بار متوالی تکرار شده است. وقتی حلقه تمام شد، اگر تعداد تکرار بیش از 1 بود، ابتدا آن عدد را به خروجی اضافه می‌کنیم (تبدیل به رشته)، سپس خود کاراکتر را اضافه می‌کنیم. اگر تکراری نبود (count برابر 1)، مستقیماً فقط کاراکتر را اضافه می‌کنیم. به این ترتیب خروجی حاوی اطلاعات فشرده‌شده خواهد بود.

کلمات کلیدی برای تشخیص این نوع سوالات عبارتند از «فشرده‌سازی» (*Komprimierung*)، «کاهش حجم داده» یا حتی «رمزگذاری ساده». در پاسخ، شما باید الگوریتمی بنویسید که رشته یا دنباله‌ای از داده‌ها را طبق قانون تعریف‌شده در سوال فشرده کند. دقت کنید که تمام جزئیات قواعد داده‌شده (مثل از چه عددی به بعد فشرده‌سازی انجام شود، یا آیا موارد بدون تکرار عیناً کپی می‌شوند) را رعایت کنید. همچنین، توجه به انتهای حلقه و به‌روزرسانی اندیس i بسیار مهم است تا از حلقه بی‌نهایت جلوگیری شود. نسخه شبه‌جاوای ارائه‌شده نشان می‌دهد که چگونه همین منطق را می‌توان در یک زبان واقعی (Java) پیاده‌سازی کرد. در امتحان لازم نیست حتماً از ساختارهایی مثل StringBuilder نام ببرید، اما نوشتن خروجی به یک رشته موقت (همان komprimiert در شبه‌کد) و الحاق (append) به آن باید واضح بیان شود.